OMP: Parallel regions

Prof. Ramiro A Barrios Valencia

Parallel region directive

Code within a parallel regino is executed by all threads

Syntax:

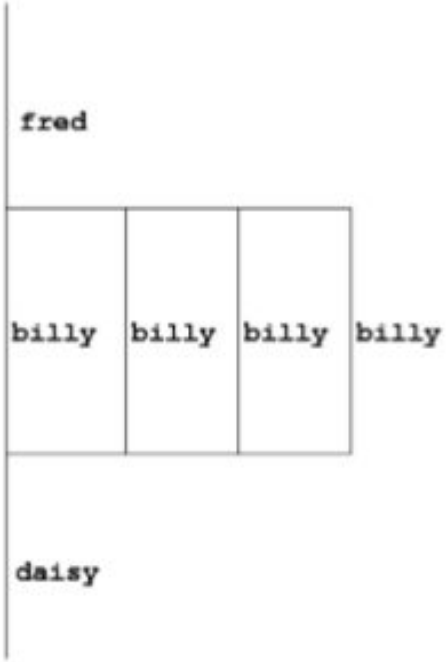
C/C++:

```
#pragma omp parallel
{
   block
}
```

Parallel region directive (cont)

Example:

```
fred();
#pragma omp parallel
{
   billy();
}
daisy();
```



Useful functions

Often useful to find out number of threads being used

C/C++:

```
#include <omp.h>
    int omp_get_num_threads(void);
```

- Important note: returns 1 if called outside parallel region!

Useful functions (cont)

Also useful to find out number of the executing threads.

C/C++:

```
#include <omp.h>
    int omp_get_thread_num(void)
```

- Takes values between 0 and `OMP_GET_NUM_THREADS()` - 1

Clauses

Specify additional information in ht eparallel region directive through clauses

Clauses are comma or space separate

- C/C++: `#pragma omp parallel [clauses]`

Shared and private variables

Inside a parallel region, variables can be eigther shared (all threads see same copy) or private (each thread has it own copy)

Shared, private and default clauses

C/C++: `shared(list)`
`private(list)`
`default(shared|none)`

Shared and private variables (cont)

On entry to a parallel region, private variables are uninitialised

Variables declared inside the scope of the parallel region are automatically private

After the parallel region ends the original variable is unaffected by any changes to private copies

in C++ private objects are created using the default constructor

Not specifying a DEFAULT clause is the same as specifying DEFAULT(SHARED)

- Danger
- Always use DEFAULT(NONE)

Multi-line directives

- C/C++:

```
#pragma omp parallel default(none) \
private(i,myid) shared(a,n)
```

Initialising private variables

Private variables are unintialised at the start of the parallel region

If we wish to initialise them, we use the FIRSTPRIVATE clause:

C/C++: **`firstprivate`** (*list*)

Use cases for this are uncommon

In C++ the default copy constructor is called to create and initialise the new object

Initialising private variables (cont)

Example:

```
    b = 23.0;
    . . . . .
#pragma omp parallel firstprivate(b), private(i,myid)
  {
    myid = omp_get_thread_num();
    for (i=0; i<n; i++){
      b += c[myid][i];
    }
    c[myid][n] = b;
  }
```

Reductions

A reduction produces a single value from associative operations such as addition, multiplication, mas, min, and, or.

Would like each thread to reduce into a private copy, then reduce all these to give final result

Use REDUCTION clause:

C/C++: `reduction(op:list)`

In C/C++, can use a special OpenMP syntax tor array sections